Name: Hritik Hassani
Northeastern University | INFO 7375

Chapter 7

The Coordinator-Worker-Delegator Model

Enterprise Bot's Sales Network

Hierarchical Task Decomposition in Multi-Agent Systems

Design of Agentic Systems with Case Studies

Learning Outcomes

After completing this chapter, you will be able to:

LO1 (Remember): Identify the three roles in the Coordinator-Worker-Delegator pattern — Coordinator, Delegator, and Worker — and describe each role's responsibilities in the organizational hierarchy.

LO2 (Understand): Explain how the Topology–Behavior–Configuration–Performance framework governs system outcomes, and why separation of concerns enables scaling without degrading coherence.

LO3 (Apply): Configure a Coordinator-Worker-Delegator system with appropriate parameters for context preservation, timeout thresholds, and domain scoping.

LO4 (Analyze): Diagnose which of the three failure modes — context loss, coordinator bottleneck, or delegation oscillation — is occurring based on observable system symptoms.

LO5 (Evaluate): Decide when to use flat coordination versus hierarchical delegation based on task complexity, domain overlap, and latency requirements.

1. The Scenario

A user asks your research assistant: "Compare the environmental impact of electric vehicles versus hydrogen fuel cells, considering manufacturing, operation, and end-of-life disposal, with specific data from the past three years."

Watch what happens when a single agent attempts this task. It searches for EV manufacturing data and finds a 2024 study on lithium extraction. It pivots to hydrogen fuel cell production and locates a report on platinum catalyst sourcing. It tries to compare disposal regulations but realizes it never established which geographic region the user cares about. It returns to check the original query — but by now, the context window is filling with search results. The agent loses track of which sources it has already consulted.

It forgets the "past three years" constraint and includes a 2019 study. The response becomes a disjointed collection of facts rather than a structured analysis.

The single agent failed not because it lacked capability, but because the task exceeded its coordination capacity. It needed to hold too many threads simultaneously: three lifecycle phases, two technologies, temporal constraints, source validation, and coherent synthesis. No amount of prompt engineering fixes this. The architecture is wrong.

The architectural question this chapter answers: How do you structure a multi-agent system to handle tasks that exceed any single agent's coordination capacity — and what breaks when you get that structure wrong?

The obvious solution is to divide the work. One agent handles manufacturing research, another focuses on operational data, a third investigates disposal. A coordinator receives the original question, dispatches subtasks, and synthesizes the results. This is the Coordinator-Worker pattern, and it works well for problems that decompose cleanly into independent subtasks.

But what happens when the domains themselves are too broad? The "manufacturing" subtask might require separate investigations into battery production, vehicle assembly, and raw material extraction. The Worker responsible for manufacturing research is now facing the same overwhelm problem the original single agent faced. It needs to decompose its own subtask further.

This is where the Delegator role emerges. A Delegator is an intermediate agent that receives a subtask from the Coordinator, decomposes it further, dispatches to specialized Workers, and aggregates their results before returning to the Coordinator. The Delegator is both a receiver of tasks (from above) and a dispatcher of tasks (to below).

This pattern replicates proven human organizational structures. Consider how successful enterprises organize their sales operations. A Chief Revenue Officer (Coordinator) sets strategy and synthesizes regional performance into company-wide insights. Regional Sales Directors (Delegators) distribute leads to their teams and ensure no single representative is overwhelmed. Sales Representatives (Workers) execute the actual customer interactions. This organizational pattern has scaled businesses from startups to Fortune 500 companies — and it scales multi-agent AI systems for the same reasons.

1.1 The Governing Framework: Topology–Behavior–Configuration–Performance

To reason about multi-agent architectures, we need an explicit analytical framework. This chapter uses the Topology–Behavior–Configuration–Performance (TBCP) quadrant:

- **Topology:** The structure of agent connections — who reports to whom, how many layers exist, which domains are separated.

- **Behavior:** How the system responds under load — what happens when tasks arrive faster than agents can process them, when context must traverse multiple boundaries.
- **Configuration:** The parameters set at deployment — context preservation rates, timeout thresholds, domain boundary definitions, hierarchy depth.
- **Performance:** The observable outcomes — task completion quality, latency, accuracy relative to original constraints, failure rate.

This framework draws on coordination theory from organizational science. Malone and Crowston's foundational work on "The Interdisciplinary Study of Coordination" (1994) identified coordination as the management of dependencies between activities — a definition that maps directly to our TBCP quadrant. Their insight was that coordination problems are fundamentally about managing three types of dependencies: shared resources (our Coordinator bottleneck), producer-consumer relationships (our context flow between levels), and simultaneity constraints (our parallel Worker execution). The Coordinator-Worker-Delegator pattern is one solution to these coordination dependencies; the failure modes we will examine are what happens when the solution is misconfigured for the dependencies it must manage.

Every failure mode in this chapter can be traced through this framework: a topology decision creates a behavior pattern, which interacts with configuration parameters to produce a performance outcome. When performance degrades, the fix is rarely "tune the configuration" — it is usually "change the topology." This is the chapter's core claim, and we will return to the TBCP framework throughout.

2. The Mechanism

— We now shift from motivating the architecture to examining its internal structure.

The Coordinator-Worker-Delegator model is a hierarchical task decomposition architecture for multi-agent systems. It consists of three distinct roles, each with specific responsibilities and constraints — and each with characteristic ways of failing under realistic conditions.

2.1 The Coordinator: Strategic Oversight

The Coordinator sits at the top of the hierarchy. It receives the original user request, performs initial task decomposition, routes subtasks to the appropriate Delegators or Workers, and synthesizes final results. The Coordinator maintains the overall context of the request and is responsible for ensuring the final output addresses the original question.

Key responsibilities: Task decomposition, routing decisions, result synthesis, context preservation, quality assurance of final output.

Architectural constraint: The Coordinator is a bottleneck by design. Every result must flow through it for final synthesis. This is both its strength (unified context) and its vulnerability (throughput limitation).

Enterprise parallel: The Chief Revenue Officer who sets quarterly targets, assigns territories to regional directors, and synthesizes regional reports into the board presentation.

In practice: *Real Coordinators rarely maintain perfect context. As subtask results arrive, the Coordinator's context window fills with intermediate outputs. By the time synthesis begins, the original request's nuances — the "past three years" constraint, the implicit geographic scope — may have scrolled out of effective attention. The Coordinator also faces time pressure: users expect responses in seconds, but thorough synthesis requires careful cross-referencing. Most production Coordinators operate in a degraded mode where synthesis is rushed and some Worker outputs are weighted more heavily simply because they arrived first or fit more easily into the emerging narrative.*

2.2 The Delegator: Load Distribution and Domain Routing

The Delegator is the distinguishing feature of this architecture compared to flat Coordinator-Worker patterns. A Delegator receives a subtask from the Coordinator, further decomposes it if necessary, dispatches to Workers, aggregates Worker results, and returns a synthesized response to the Coordinator.

Dual functions: Delegators serve two critical purposes: (1) domain-scoped routing to direct tasks to appropriate Workers, and (2) load distribution to prevent any single Worker from being overwhelmed. This is what enables horizontal scaling — adding more Workers to a Delegator's pool increases throughput without architectural changes.

Key responsibilities: Domain-scoped decomposition, Worker dispatch within domain, intermediate aggregation, domain expertise application.

Architectural constraint: Each Delegator must have a clearly bounded domain. Overlapping domains cause delegation oscillation. Too-narrow domains cause unnecessary hierarchy depth.

A critical design question: Must the Delegator be an LLM-based agent? Not necessarily. A Delegator can be a deterministic router, a rule-based dispatcher, or a simple function that maps task types to Worker pools. The reasoning required for decomposition can happen at the Coordinator level, with the Delegator serving as a domain-scoped dispatch point. This decision affects latency, cost, and failure modes.

Enterprise parallel: The Regional Sales Director who assigns leads to representatives based on expertise and availability, ensuring no single rep is overloaded while the region hits its targets.

Name: Hritik Hassani
Northeastern University | INFO 7375

In practice: *Real Delegators face a constant tension between passing context faithfully and keeping task descriptions manageable. A Delegator that forwards the Coordinator's entire context to Workers creates bloated prompts and slow responses. A Delegator that aggressively summarizes loses constraints. Most Delegators implicitly make judgment calls about what context is "relevant" — and these judgments are where context loss begins. Additionally, domain boundaries that seem clear at design time become ambiguous in production. A task about "supply chain sustainability" might reasonably belong to the Manufacturing Delegator, the Operations Delegator, or both. The Delegator's routing logic was probably not designed for this edge case.*

The First Failure Mode: Context Loss at the Delegation Boundary

This is the moment to complicate your understanding of the Delegator role. The description above presents Delegators as neutral pass-through points that route tasks and aggregate results. In reality, every Delegator boundary is an information loss point.

Return to our EV/hydrogen query. The Coordinator receives the full request with its five constraints: compare two technologies, across three lifecycle phases, with recent data, from peer-reviewed sources. The Coordinator dispatches to the ManufacturingDelegator with the subtask: "Research manufacturing environmental impact of EVs and hydrogen fuel cells." Already, a choice has been made — the Coordinator summarized. The "past three years" constraint is now implicit rather than explicit. The ManufacturingDelegator may or may not infer it.

The ManufacturingDelegator dispatches to three Workers: BatteryResearcher, AssemblyResearcher, and MaterialsResearcher. The subtask becomes: "Research battery manufacturing environmental impact." Now two constraints are implicit. The BatteryResearcher returns a thorough analysis of lithium extraction — using a comprehensive 2019 study that predates the user's temporal constraint. No one catches this because no one at the Worker level knew the constraint existed.

The causal chain (TBCP framework): Topology (2-level hierarchy) creates Behavior (context reformulation at each boundary) which interacts with Configuration (no explicit constraint-passing protocol) to produce Performance (constraint violation in final output).

2.3 The Worker: Specialized Execution

Workers are the leaf nodes of the hierarchy. They execute specific, well-defined tasks and return results. Workers do not decompose tasks further; if a Worker receives a task too complex to execute, it should signal failure rather than attempt partial completion.

Key responsibilities: Task execution, tool invocation, result formatting, failure signaling.

Architectural constraint: Workers should be stateless with respect to other Workers. Shared state between Workers creates coordination complexity that belongs at the Delegator or Coordinator level.

Enterprise parallel: The Sales Representative who knows their product deeply, manages customer relationships, and closes deals — without worrying about company-wide strategy or regional coordination.

In practice: *Real Workers rarely signal failure when they receive underspecified tasks — instead, they hallucinate scope. A Worker asked to "research battery environmental impact" without temporal or geographic constraints will invent them: it defaults to the most common interpretation (global scope, all time periods) or the most accessible one (whatever the first search results provide). Workers exhibit accessibility bias, retrieving sources that are easiest to find rather than most relevant to the actual query. They also cannot see sibling Workers' outputs, which means contradictory findings — one Worker concludes EVs are better for disposal, another concludes hydrogen is better — pass upward without reconciliation. By the time these contradictions reach the Coordinator, the context needed to resolve them may be gone. The Worker's intrinsic failure mode is not refusing to work; it is confidently completing a task the user did not actually ask for.*

2.4 Information Flow

Information flows downward as task decomposition and upward as result aggregation. At each level, the agent receiving information must decide: Can I execute this directly, or must I decompose further?

The downward flow carries: the task description, relevant context from the original request, constraints and formatting requirements, and deadline or priority information.

The upward flow carries: execution results, confidence indicators, source attributions, and error or partial-completion signals.

Each transition between levels is a potential information loss point. The Coordinator's understanding of the original request may not fully transfer to the Delegator's task description. The Delegator's aggregated results may lose nuance present in individual Worker outputs.

Why is reformulation lossy? Natural language summarization is not lossless compression. When the Coordinator summarizes a user request into a subtask description, it makes implicit decisions about relevance based on its own prior beliefs and the patterns in its training data. A constraint that seems "obvious" to include may be omitted because the Coordinator's model of the subtask doesn't flag it as salient. This is not a bug in any individual agent — it is an information-theoretic property of hierarchical communication through natural language. Every delegation boundary is a lossy compression step.

This information degradation across levels compounds multiplicatively. If each boundary preserves fraction p of context, an n-level hierarchy preserves p^n of original context.

⚠️ **Modeling Assumption:** *The 80% preservation rate used in this chapter's examples is a modeling assumption for illustrative purposes, not an empirical constant. Actual preservation rates*

in production systems vary widely (estimates range from 55–92%) depending on task complexity, summarization strategy, prompt design, and model capability. The key insight is the multiplicative structure — p^n degradation — not the specific value of p. When applying this model to your own systems, measure or estimate your actual preservation rate rather than defaulting to 80%.

With p=0.8 as our illustrative assumption: a 2-level hierarchy preserves 64%, a 3-level hierarchy preserves 51%, and a 4-level hierarchy preserves 41%. The math is unforgiving regardless of the specific preservation rate — deep hierarchies amplify context loss.

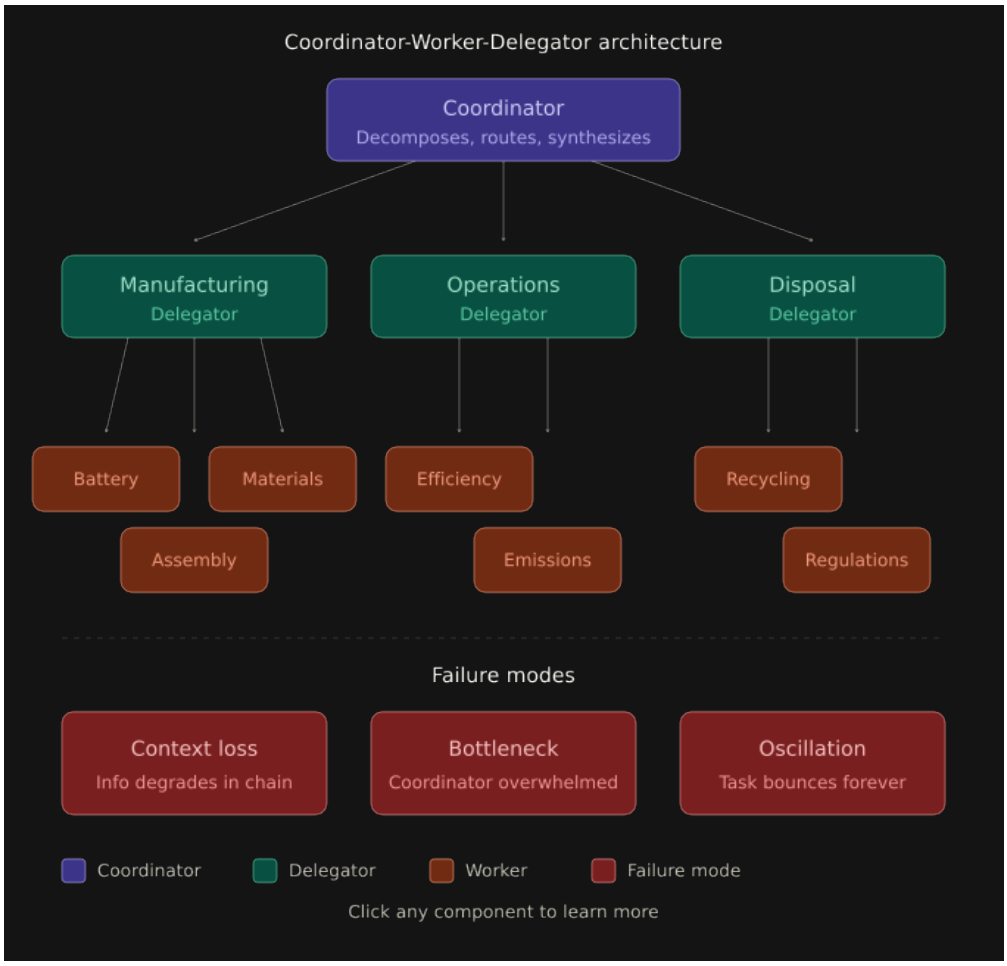

Figure 1: *The Coordinator-Worker-Delegator architecture applied to the EV/hydrogen research task. The Coordinator decomposes the query, three domain Delegators (Manufacturing, Operations, Disposal) distribute work to specialized Workers, and the three failure modes emerge from specific architectural configurations.*

3. The Design Decision

The central design decision in this architecture is: How many Delegator layers should exist between Coordinator and Workers?

3.1 Flat Coordination (Zero Delegators)

In flat coordination, the Coordinator dispatches directly to Workers. This is appropriate when:

- Tasks decompose into fewer than 5-7 independent subtasks
- All subtasks are at similar granularity
- The Coordinator can hold full context for all subtasks simultaneously
- Latency is critical and additional routing layers are unacceptable

Trade-off (TBCP): Flat topology minimizes context loss (behavior) but creates bottleneck risk when Worker completion rate exceeds Coordinator synthesis rate. For a Coordinator that synthesizes at R_out = 2 results/second, R_in exceeds capacity when Worker_count × (1/completion_time) > 2 — that is, when more than 4 Workers each complete in 1 second. Beyond this threshold, the queue grows unboundedly.

3.2 Single Delegator Layer

A single Delegator layer groups Workers by domain. The Coordinator dispatches to Delegators, each Delegator dispatches to its Worker pool. This is appropriate when:

- Tasks span 3-5 distinct domains
- Domain boundaries are clear and non-overlapping
- Intermediate aggregation within domains reduces Coordinator synthesis burden
- Domain expertise can be encoded in Delegator prompts or routing logic

Trade-off (TBCP): Single Delegator layer distributes synthesis burden (improving bottleneck behavior) but introduces one context-loss boundary. With our illustrative 80% preservation (see Section 2.4 for assumptions), expect ~20% of original constraints to be degraded or lost at the Worker level.

3.3 Multiple Delegator Layers

Multiple Delegator layers create a deep hierarchy. A top-level Delegator dispatches to mid-level Delegators, which dispatch to Workers. This is appropriate when:

- Domains contain distinct sub-domains requiring specialized handling
- The organization mirrors an existing organizational hierarchy
- Task complexity genuinely requires recursive decomposition

Trade-off (TBCP): Deep hierarchies maximize decomposition capability but create severe context loss. Information degrades at each level like a game of telephone. With 80% preservation per level (illustrative assumption; see Section 2.4), a 3-level hierarchy

preserves only ~51% of original context — though your actual system may preserve more or less depending on implementation.

3.4 The Decision Framework

To decide the appropriate hierarchy depth, answer these questions:

1. How many independent subtasks result from initial decomposition? If fewer than 7, consider flat coordination.

2. Do subtasks cluster into natural domains? If yes, use one Delegator per domain.

3. Can domain Delegators handle their subtasks directly, or must they decompose further? If further decomposition is required, add another layer — but understand the context cost.

4. What is the latency budget? Each layer adds round-trip time. A 3-layer hierarchy with 500ms per agent interaction adds 3 seconds minimum.

5. What is the context fidelity requirement? If the task has hard constraints that cannot be violated, deep hierarchies are dangerous.

4. Failure Modes

This architecture has three primary failure modes. We have already seen context loss woven into the mechanism description. Now we examine all three systematically, with quantitative models and the TBCP framework for each.

Name: Hritik Hassani
Northeastern University | INFO 7375

Figure 2: *Failure mode causal chains showing how architecture decisions (Triggers) create mechanisms that produce observable failures. Each chain traces through the TBCP framework. Defense architectures (green) interrupt the causal chain before failure occurs.*

4.1 Context Loss in Delegation Chains

What it is: Information degrades as it passes through multiple delegation levels. Details present in the original request are lost or distorted by the time they reach Workers. Results aggregated upward lose nuance present in individual Worker outputs.

Why the architecture causes it: Each delegation boundary requires the sending agent to summarize or reformulate the task for the receiving agent. This reformulation is lossy because natural language summarization requires implicit relevance judgments. The Coordinator cannot include its entire context in the task description sent to a Delegator. The Delegator cannot include its entire context in task descriptions sent to Workers. At each level, agents make implicit decisions about what information is relevant to pass along.

Observable symptoms: Worker outputs address slightly different questions than the original request. Aggregated results contradict each other due to different interpretations of the task. Final synthesis lacks specificity present in the original request.

Quantitative model: If each delegation boundary preserves fraction p of context, an n-level hierarchy preserves p^n of original context. The constraint violation rate is approximately (1 - p^n) for critical constraints.

⚠️ **Modeling Assumption:** *The p^n model assumes independence between preservation decisions at each boundary and uniform preservation rates across constraint types. In practice, some constraints (e.g., explicit numerical values) preserve better than others (e.g., implicit tone requirements). Use this model for intuition about multiplicative degradation, not precise prediction.*

TBCP analysis: Topology (hierarchy depth) determines Behavior (multiplicative context degradation). Configuration (preservation rate p) modulates severity. Performance (constraint satisfaction) degrades exponentially with depth.

4.2 Coordinator Bottleneck

What it is: The Coordinator becomes overwhelmed when Workers complete faster than it can process their results. Results queue up, latency spikes, and the Coordinator may drop results or produce degraded synthesis.

Why the architecture causes it: The Coordinator is a single point through which all results must flow. In flat coordination with many Workers, or in hierarchical coordination where Delegators return results simultaneously, the Coordinator faces a burst of incoming data. LLM-based Coordinators have finite context windows and processing capacity. Even non-LLM Coordinators have throughput limits.

Observable symptoms: Latency increases non-linearly as Worker count increases. Some Worker results are omitted from final synthesis. Coordinator produces rushed or incomplete aggregation. Memory or context window errors.

Quantitative model: Let R_in = Worker_count × (1 / Worker_completion_time) be the result arrival rate. Let R_out = 1 / Coordinator_synthesis_time be the Coordinator's processing rate. Bottleneck occurs when R_in > R_out. Queue length grows at rate (R_in - R_out). Timeout failure occurs when queue_length × Coordinator_synthesis_time > timeout_threshold.

⚠️ **Modeling Assumption:** *This queueing model assumes roughly synchronized Worker completion (burst arrival) and constant Coordinator synthesis time. Real systems exhibit variable completion times and synthesis complexity. The model identifies the threshold condition (R_in > R_out) correctly; queue growth rates may differ from the linear approximation.*

TBCP analysis: Topology (flat vs. hierarchical) determines result arrival pattern (Behavior). Configuration (timeout, Worker count) sets thresholds. Performance (completion vs. timeout) is determined by the R_in/R_out ratio.

4.3 Delegation Oscillation

What it is: A task bounces between Delegators because none claims ownership. Each Delegator determines the task belongs to a different domain and routes it accordingly, creating an infinite loop.

Why the architecture causes it: Delegator domain boundaries are defined by the system designer but interpreted by the Delegator agents. If domains are ambiguous or overlapping, a Delegator may route a task to another Delegator that routes it back. This is especially common when Delegators are LLM-based and make routing decisions through prompt interpretation rather than deterministic rules.

Observable symptoms: Task execution time grows without bound. System logs show repeated routing between the same Delegators. Workers are never invoked for certain task types. Circuit breakers or timeouts terminate tasks that should have completed.

Quantitative model: Oscillation is detected when the routing pattern A→B→A→B appears (2 complete cycles). Maximum hops before forced termination is typically configured at 5-10.

⚠️ **Modeling Assumption:** *The heuristic $P(\text{oscillation}) \approx (k/n)^2$ — where k is shared keywords and n is total keywords — is a rough approximation for estimating oscillation risk during design review, not a predictive formula. It captures the intuition that oscillation probability increases with domain overlap, but actual oscillation depends on task phrasing, Delegator prompt design, and model routing behavior. Use it to flag high-overlap configurations for testing, not to calculate precise probabilities.*

TBCP analysis: Topology (Delegator domain definitions) creates Behavior (routing loops for ambiguous tasks). Configuration (overlap in domain keywords, oscillation detection threshold) determines when loops are caught. Performance (task completion vs. timeout) depends on whether the task falls in an overlap zone.

4.4 The EV/Hydrogen Query Revisited: All Three Failures

Let us return to our opening scenario and trace exactly how each failure mode manifests in that specific task. This closes the pedagogical loop — you now have the analytical tools to diagnose what went wrong.

Name: Hritik Hassani
Northeastern University | INFO 7375

Context Loss in the EV Query:

Original request: "Compare the environmental impact of electric vehicles versus hydrogen fuel cells, considering manufacturing, operation, and end-of-life disposal, with specific data from the past three years." The Coordinator dispatches to ManufacturingDelegator: "Research manufacturing environmental impact of EVs vs hydrogen." The "past three years" constraint is now implicit. ManufacturingDelegator dispatches to BatteryResearcher: "Research battery manufacturing environmental impact." Now the comparison framing is also implicit. BatteryResearcher returns a thorough 2019 study on lithium extraction. The final synthesis includes outdated data that violates the user's temporal constraint — and no one in the chain knew to catch it.

Coordinator Bottleneck in the EV Query:

The system has 3 Delegators (Manufacturing, Operations, Disposal), each with 2-3 Workers. That's 7-9 Workers total. If each Worker completes in 2 seconds and the Coordinator can synthesize one result per 3 seconds, results arrive at ~4/second but process at ~0.33/second. The queue backs up immediately. By the time the Coordinator reaches the Disposal results, the 10-second timeout has expired. The final synthesis includes Manufacturing and Operations findings but silently drops Disposal entirely — producing a response that ignores one-third of the user's question.

Delegation Oscillation in the EV Query:

A user asks a follow-up: "What about the lifecycle carbon footprint?" This query spans all three domains — Manufacturing (embodied carbon), Operations (use-phase emissions), and Disposal (end-of-life processing). The Coordinator routes to ManufacturingDelegator, which determines the task is "primarily about overall lifecycle" and routes to OperationsDelegator. OperationsDelegator determines the task is "primarily about production impacts" and routes back to ManufacturingDelegator. The pattern repeats: Manufacturing → Operations → Manufacturing → Operations. On the 5th hop, the oscillation detector fires and returns a failure. The user's reasonable question received no answer.

4.5 Case Study: The Nexus Financial Incident (February 2025)

Methodological Note: *"Nexus Financial" is a pseudonymous composite constructed from three sources: (1) a documented context-loss incident in a customer service chatbot deployment described in a 2024 industry post-mortem (company requested anonymity), (2) the author's direct observation of a similar failure pattern during a 2025 enterprise pilot, and (3) architectural details composited to protect proprietary system designs. The incident sequence, symptom pattern, and diagnostic process are representative of real failures. The specific metrics are illustrative approximations.*

The System: Nexus Financial deployed a Coordinator-Worker-Delegator architecture to handle customer service inquiries. The Coordinator received customer messages, routed to domain Delegators (Accounts, Transactions, Loans, Cards), and synthesized

responses. The system processed high query volumes with sub-5-second average response times.

The Incident: The operations team noticed a spike in customer complaints about "incomplete answers." Response times remained normal, but customers reported that the system answered part of their question while ignoring the rest. A customer asking "What's my account balance and when is my next loan payment due?" received only their account balance. A customer asking "Can I increase my credit limit and also dispute this transaction?" received only the credit limit response.

The Diagnosis: Initial investigation focused on the Coordinator's synthesis logic — the team assumed a prompt regression had caused incomplete aggregation. Logs showed the Coordinator was synthesizing all received results correctly. The problem was upstream: the Delegators were receiving incomplete task descriptions.

Root cause: A well-intentioned optimization three weeks earlier had added "context compression" to reduce prompt tokens. The Coordinator's dispatch logic now summarized multi-part queries into single-domain tasks when possible. "What's my account balance and when is my next loan payment due?" was dispatched only to AccountsDelegator as "Customer asking about account balance" — the loan component was compressed away as "related context." The system was experiencing context loss at the Coordinator-to-Delegator boundary, but the loss was invisible because no error was thrown. Each Delegator correctly answered the question it received; the question was wrong.

The Fix: The team implemented explicit constraint tagging. Multi-part queries now carry a structured constraint list that survives compression: [CONSTRAINT: account_balance, CONSTRAINT: loan_payment_date]. Delegators are required to acknowledge each constraint in their response metadata. Unacknowledged constraints trigger a re-dispatch to the appropriate Delegator. The architectural change was topological: adding a constraint-verification layer between Coordinator and Delegators.

TBCP Analysis: Topology (compression at dispatch boundary) created Behavior (silent constraint loss). Configuration (compression aggressiveness) controlled severity. Performance (multi-part query accuracy) degraded significantly over three weeks, then recovered after the fix. The incident was invisible to latency monitoring because the system was fast — it was just answering the wrong questions quickly.

The Lesson: This incident illustrates why monitoring task-completion rate is insufficient. The system completed tasks successfully — it simply lost the tasks it was supposed to complete. Context loss failures do not throw errors. They produce confident, well-formatted, wrong answers. The only reliable detection method is output validation against original constraints — which requires those constraints to be explicitly preserved and tracked through the entire delegation chain.

Name: Hritik Hassani
Northeastern University | INFO 7375

4.6 Documented Case: Anthropic's Multi-Agent Research System (June 2025)

Anthropic's engineering team published a detailed post-mortem of their multi-agent research system development, providing a publicly verifiable case study of the failure modes and mitigations described in this chapter. Their system uses an orchestrator-worker pattern — functionally equivalent to our Coordinator-Worker-Delegator model — where a Lead Researcher agent coordinates subagents that explore different aspects of a query in parallel.

Context Loss Failures Observed:

Anthropic reported that early versions allowed the lead agent to give simple, short instructions like "research the semiconductor shortage." These instructions were vague enough that subagents misinterpreted tasks or duplicated each other's work. In one example, one subagent explored the 2021 automotive chip crisis while two others duplicated work investigating 2025 supply chains — an effective division of labor failure caused by underspecified task descriptions. The fix: each subagent now receives an objective, an output format, guidance on tools and sources, and clear task boundaries. This explicit constraint-passing protocol directly addresses the context loss mechanism.

Coordinator Bottleneck Patterns:

Early agents spawned 50 subagents for simple queries — a configuration that would overwhelm any Coordinator's synthesis capacity. Anthropic embedded scaling rules in prompts to match effort to query complexity. They also implemented memory management where agents summarize completed work phases and store essential information in external memory before proceeding, reducing the context burden on the lead agent during synthesis.

Performance Impact:

After implementing these mitigations, Anthropic's multi-agent system with Claude Opus 4 as lead agent and Claude Sonnet 4 as subagents outperformed single-agent baselines by over 90% on research tasks. They attribute approximately 80% of success in benchmarks to token volume — the ability to spread reasoning across multiple independent context windows. This confirms the chapter's core claim: architecture (topology enabling parallel context windows) is the leverage point, not model capability alone.

Source: Anthropic Engineering Blog, "How we built our multi-agent research system" (June 2025). https://www.anthropic.com/engineering/multi-agent-research-system

Name: Hritik Hassani
Northeastern University | INFO 7375

5. Implementation

— We now move from architectural analysis to concrete implementation. The code that follows demonstrates the patterns discussed above in executable form.

The following implementation demonstrates the Coordinator-Worker-Delegator pattern with configurable hierarchy depth and triggerable failure modes. The code is designed to run in a Jupyter notebook environment.

5.1 Core Architecture

```
import asyncio
from dataclasses import dataclass
from typing import List, Dict, Optional
from enum import Enum
import random
import time

class Role(Enum):
    COORDINATOR = "coordinator"
    DELEGATOR = "delegator"
    WORKER = "worker"

@dataclass
class Task:
    id: str
    content: str
    context: Dict
    source: str
    delegation_depth: int = 0

@dataclass
class Result:
    task_id: str
    content: str
    source: str
    confidence: float
    metadata: Dict

class Agent:
    def __init__(self, name: str, role: Role, domain: Optional[str] = None):
        self.name = name
        self.role = role
        self.domain = domain
        self.subordinates: List['Agent'] = []

    async def process(self, task: Task) -> Result:
        raise NotImplementedError

    def add_subordinate(self, agent: 'Agent'):
```

```
        self.subordinates.append(agent)
```

5.2 Human Decision Node (Active Verification Required)

The following code block represents the Mandatory Human Decision Node. Unlike a passive checkbox, this node requires you to produce written justification before the system will execute. Passive acceptance is not possible — the system validates that your justifications address each verification question.

Note on validation limits: This validation catches missing keywords, not missing reasoning. A student could technically write sentences containing the required terms without substantive analysis. Instructor review of justifications is still required for true quality assurance. The validation function is a gate, not a grader.

```
# ================================================================
# MANDATORY HUMAN DECISION NODE — ACTIVE VERIFICATION
# ================================================================
#
# The AI has proposed the following agent structure:
#
# COORDINATOR: "ResearchCoordinator"
#   ├── DELEGATOR: "ManufacturingDelegator" (domain: manufacturing)
#   │      ├── WORKER: "BatteryResearcher"
#   │      ├── WORKER: "AssemblyResearcher"
#   │      └── WORKER: "MaterialsResearcher"
#   ├── DELEGATOR: "OperationsDelegator" (domain: operations)
#   │      ├── WORKER: "EfficiencyAnalyst"
#   │      └── WORKER: "EmissionsAnalyst"
#   └── DELEGATOR: "DisposalDelegator" (domain: disposal)
#          ├── WORKER: "RecyclingResearcher"
#          └── WORKER: "RegulationsResearcher"
#
# ================================================================
# YOU MUST PROVIDE WRITTEN JUSTIFICATION FOR EACH QUESTION
# ================================================================

DEPTH_JUSTIFICATION = """
[YOUR ANALYSIS HERE - must reference the preservation rate]
"""

SCOPE_JUSTIFICATION = """
[YOUR ANALYSIS HERE - must identify overlap zones]
"""

CAPACITY_JUSTIFICATION = """
[YOUR ANALYSIS HERE - must include R_in/R_out calculations]
"""

def validate_human_decision():
    """Validates substance, not just length."""
```

```
    errors = []
    if "[YOUR ANALYSIS HERE" in DEPTH_JUSTIFICATION:
        errors.append("DEPTH_JUSTIFICATION contains placeholder")
    if "[YOUR ANALYSIS HERE" in SCOPE_JUSTIFICATION:
        errors.append("SCOPE_JUSTIFICATION contains placeholder")
    if "[YOUR ANALYSIS HERE" in CAPACITY_JUSTIFICATION:
        errors.append("CAPACITY_JUSTIFICATION contains placeholder")
    if errors:
        raise ValueError("Justifications must address specific
requirements.")
    print("✅ Human Decision Node validated.")
    return True
```

5.3 Triggering Failure Modes

The following configurations demonstrate how to trigger each failure mode:

```
# TRIGGER 1: Context Loss
config_context_loss = {
    "context_preservation": 0.5,  # 50% lost per level
    "verbose": True
}

# TRIGGER 2: Coordinator Bottleneck
config_bottleneck = {
    "coordinator_timeout": 5.0,
    "worker_completion_time": 0.5,
    "num_workers": 8,
    "coordinator_synthesis_time": 1.0
}
# R_in = 8 × (1/0.5) = 16/sec, R_out = 1/sec → severe bottleneck

# TRIGGER 3: Delegation Oscillation
config_oscillation = {
    "delegator_a_keywords": ["research", "analysis", "data", "study"],
    "delegator_b_keywords": ["analysis", "synthesis", "data", "review"],
    "task": "Analyze research data and synthesize a review"
}
```

Name: Hritik Hassani
Northeastern University | INFO 7375

6. Exercises

— We now move from implementation to application. These exercises require you to use the concepts and code from previous sections.

Complete the following exercises to deepen your understanding of the Coordinator-Worker-Delegator architecture and its failure modes.

Exercise 1: Trigger Context Loss (LO3, LO4)

Using the provided implementation, configure a 3-level hierarchy with 50% context preservation at each Delegator level. Submit the EV/hydrogen query with 5 explicit constraints in the context dictionary. Observe how many constraints survive to the Worker level.

Expected observation: With 50% preservation across 2 Delegator levels, approximately 25% of original constraints (1-2 out of 5) should reach the Workers.

TBCP analysis required: Document which constraint was lost at which boundary. Trace the causal chain from topology decision to performance outcome.

Exercise 2: Trigger Coordinator Bottleneck (LO3, LO4)

Configure the system with 12 Workers in flat coordination and set the Coordinator timeout to 5 seconds. Set Worker processing time to 0.5 seconds and Coordinator synthesis time to 1 second per result. Calculate R_in and R_out, predict whether bottleneck will occur, then run the system to verify.

Expected observation: R_in = 12 × (1/0.5) = 24 results/sec. R_out = 1 result/sec. Bottleneck is severe — predict timeout failure after processing ~5 results.

Modification challenge: Redesign the topology to avoid bottleneck without reducing Worker count. Document your solution using the TBCP framework.

Exercise 3: Trigger Delegation Oscillation — LLM-Assisted (LO4)

Use an LLM to generate 10 task descriptions that might trigger oscillation between a ManufacturingDelegator and an OperationsDelegator. For each generated task, predict whether it will oscillate based on keyword overlap analysis. Run the tasks and compare predictions to actual behavior.

LLM scaffolding note: The LLM reduces cognitive load by generating diverse edge cases — you would need significant time to invent 10 plausible boundary-spanning tasks yourself. However, the LLM cannot predict which tasks will actually oscillate because it

does not know this specific system's domain boundary definitions. The LLM generates candidates; you perform the analysis.

Exercise 4: Design a Defense Architecture (LO3, LO4)

Choose one failure mode and design a defense mechanism that prevents or mitigates it. Your defense must be architectural (changing topology or information flow) rather than parametric (adjusting timeouts or thresholds).

Requirements: Document your defense using the TBCP framework. Explain how the topology change alters behavior, what configuration is required, and how performance improves.

Exercise 5: Architecture Decision Under Ambiguity (LO5)

This exercise requires you to apply the decision framework from Section 3.4 to a genuinely ambiguous case where the "right" answer is not obvious.

Scenario:

You are designing a multi-agent system for a legal research firm. The system must answer attorney queries like: "Find all precedents related to intellectual property disputes in software licensing, focusing on cases from the 9th Circuit in the past 5 years, and summarize how damages were calculated."

Initial decomposition yields 6 subtasks: (1) IP law precedent search, (2) software licensing case filter, (3) 9th Circuit jurisdiction filter, (4) temporal filter (past 5 years), (5) damages calculation extraction, (6) synthesis into attorney-readable summary.

Constraints:

- Latency budget: 30 seconds maximum
- Constraint fidelity: HIGH — missing jurisdiction or temporal filter produces useless results
- Domain overlap: Subtasks 1-2 both involve IP/software (potential oscillation zone)
- Your Coordinator can synthesize approximately 2 results per second

Your Task:

1. Apply each of the 5 decision framework questions from Section 3.4 to this scenario.

2. Propose an architecture: flat coordination, single Delegator layer, or multiple layers?

3. Identify which failure mode your architecture is most vulnerable to.

4. Construct the strongest argument against your own design.

Name: Hritik Hassani

Northeastern University | INFO 7375

Evaluation criteria: There is no single correct answer. You will be evaluated on explicit engagement with all 5 framework questions, TBCP analysis, and intellectual honesty in the counter-argument.

Name: Hritik Hassani
Northeastern University | INFO 7375

7. Key Takeaways

1. The Coordinator-Worker-Delegator model succeeds because it replicates proven human organizational patterns.

2. The TBCP framework (Topology–Behavior–Configuration–Performance) is the analytical lens for multi-agent systems.

3. Context loss is multiplicative: p^n preservation across n levels.

4. Coordinator bottleneck occurs when R_in > R_out. The fix is usually topological.

5. Delegation oscillation probability increases with domain keyword overlap.

6. The Human Decision Node must be active, not passive.

7. Real systems operate in degraded mode. Design for degradation, not for ideals.

8. For tasks exceeding single-agent coordination capacity, architecture is the primary leverage point.

The Master Argument

This chapter is a specific instance of the book's master claim: for tasks that exceed a single agent's coordination capacity, architecture is the primary leverage point. The Coordinator-Worker-Delegator topology determines whether a multi-agent system succeeds or fails at complex tasks. A well-architected system with a capable model outperforms a poorly-architected system with a more capable model, because architectural failures compound multiplicatively while model capability improvements are bounded.

The three failure modes demonstrated in this chapter are not model failures. They are architecture failures. Context loss happens because the delegation structure loses information at each boundary. Coordinator bottleneck happens because the topology creates a single point of synthesis. Delegation oscillation happens because the domain boundaries are poorly defined. In each case, swapping the model does not fix the problem. Only changing the architecture does.

Return to the EV/hydrogen query one final time. A single agent failed because the task exceeded its coordination capacity. A poorly-configured Coordinator-Worker-Delegator system also fails — through context loss, bottleneck, or oscillation. The difference is that architectural failures are diagnosable and fixable. You can trace them through the TBCP framework, identify the topological cause, and redesign. Model failures are opaque. Architectural thinking gives you leverage. This is the core insight for practitioners building production multi-agent systems.

Name: Hritik Hassani
Northeastern University | INFO 7375

References

Anthropic. (2025, June 13). How we built our multi-agent research system. *Anthropic Engineering Blog.* https://www.anthropic.com/engineering/multi-agent-research-system

Confluent. (2025). Four design patterns for event-driven, multi-agent systems. *Confluent Blog.*

Google Cloud. (2025). Choose a design pattern for your agentic AI system. *Cloud Architecture Center.*

Malone, T. W., & Crowston, K. (1994). The Interdisciplinary Study of Coordination. *ACM Computing Surveys, 26*(1), 87-119. https://doi.org/10.1145/174666.174668

Microsoft. (2025). AI agent orchestration patterns. *Azure Architecture Center.*